

## Agentic RAG

## Simple RAG

٧٤

## Query

## Retrieval



LLM

↳ Answer

Simple GenAE

RAG  $\rightarrow$  Retrieval Augmented Generation

langchain  
langgraph

— Agents State  $\rightarrow$  Node Based Implementation

## Agentic RA<sub>n</sub>

↳

## Quem

## Agew

## Retrieval

Multiple  
(Enough Information)

Generate Answer

## Reflected Plan

Use tools

- Agentic RAG

↳ More, Dynamic

→ Decision workflow

→ Reasoning

- Reasoning
- Multiple Invoke → Tools

## Simple RAG

```

app/
├── rag/
│   ├── retriever.py ✓
│   ├── generator.py ✓
│   └── pipeline.py ✓ # Traditional RAG
└──

```

→ fixed

```

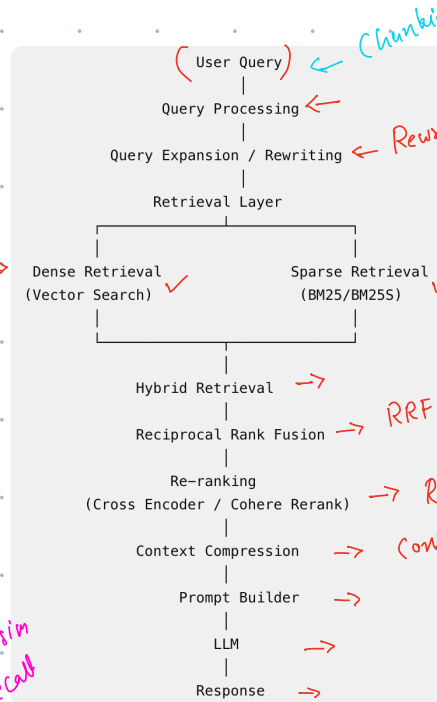
├── agentic_rag/
│   ├── agent.py ✓ # Planner/decision maker
│   ├── tools.py ✓ # Search, DB, APIs
│   ├── retriever.py ✓
│   ├── memory.py ✓
│   └── workflow.py ✓ # Agentic orchestration
└──

```

→ Flexibility  
Dynamic

HyDE  
Hypothetic Document  
Embedding

Dense



Evaluation Metric

RAHAS, Deep Eval  
→ Today's Demo

" Graph RAG "

" Vectorless RAG "

.h

## Data Ingestion

7 Different documents

- ① Document loader
- ② Chunking
- ③ Embedding model
- ④ Vector database
- ⑤ Retrieval

Bert, all mini lm lvr2, openAI Embedding & small

Pinecone  
Qdrant, Milvus

HNSW  
IVF  
Flat Indexing  
IVF PQ

## Document loaders

### Structured data

- SQL
- PostgreSQL
- MySQL
- BigQuery

### Semi structured

- JSON
- XML
- HTML
- log files

### Unstructured RAG

- ✓ PDF
- ✓ DOCX
- ✓ PPT
- ✓ MD
- ✓ Research papers

### Dynamic Data

- Kafka streams
- IoT sensors
- Slack channels

## Document loaders

### Cloud Storage

Examples:

- Amazon S3
- Google Cloud Storage
- Azure Blob Storage

### File-based loaders

- PDF Loader
- Text Loader
- CSV Loader
- JSON Loader
- DOCX Loader
- PowerPoint Loader
- Markdown Loader

### Web Loaders

These can ingest content from websites.

Examples:

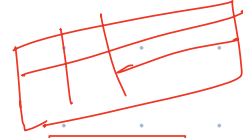
- HTML pages
- Documentation websites
- Blogs
- News articles

## Database Loaders

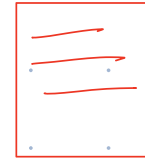
Examples:

- PostgreSQL
- MongoDB
- Redis
- Elasticsearch

Tabular



Docu  
Pres



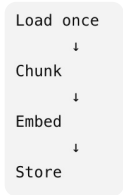
Hama Index  
Reader

- PDFs
- SQL databases
- GitHub repositories
- Google Drive
- SharePoint
- Notion
- Confluence
- Slack
- Discord
- Jira
- S3
- Azure
- Local files
- APIs

Resources  
↓

Service  
↳ Static → Dynamic

State



→ 100%

→ Knowledge Drift

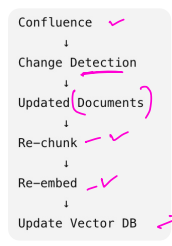


→ ✓



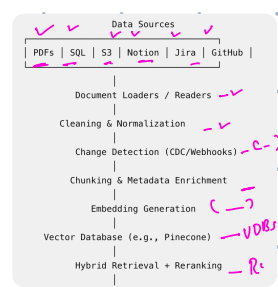
→ Reliability ↓

Synchronization  
Pipeline



→ Generation → Reliable

Production



↳ Answer

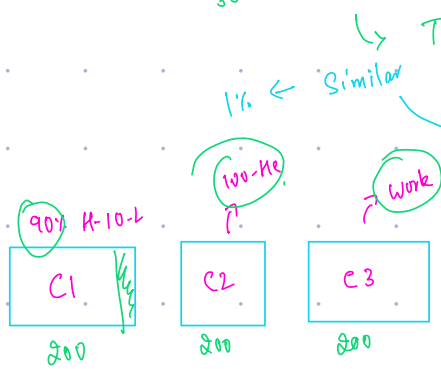
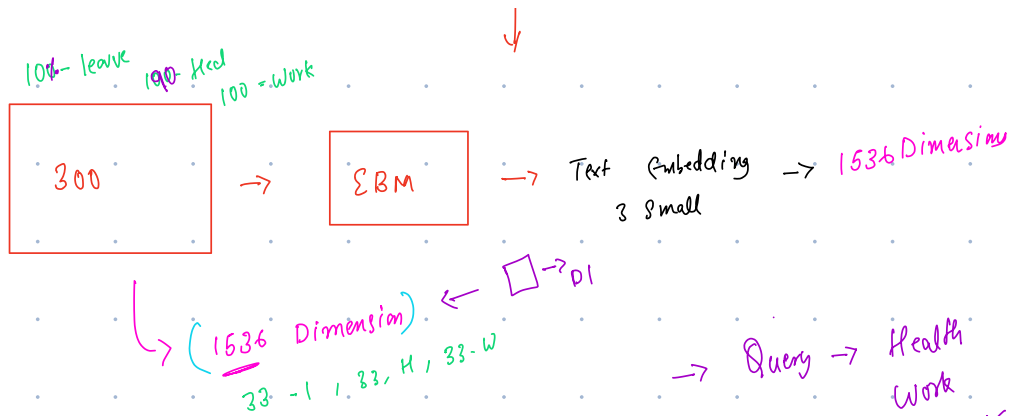


↓

Chunking



|



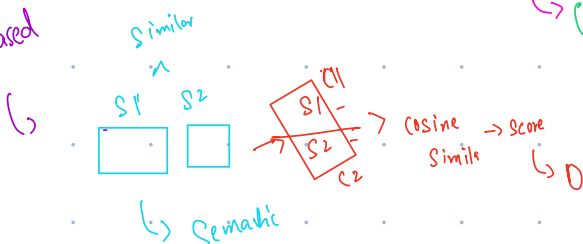
- ① fixed sized - 200 size
- ② fixed sized - overlap

### ③ Sentence Based

↳ Sentence -1  
-2  
-3

④ Recursive Text Splitter → 80% Rag

### ⑤ Semantic Based



Size = 300

↳ Paragraph - [ 200+ ] - ①  
↳ Sentence - [ 200 ] [ 200 ]  
↳ Words - [ ]  
↳ Tokens - [ ]

Size - (200 - 1000)  
↳ Too large  
↳ Too small

... ~ chunks

Embedding Models  
 Nvidia  
 OpenAI

→ Vectors

ada 2 - 1536  
 Text embedding 3 small - 1536  
 BGE - 384  
 All mini lm - 384  
 llm - 384

flugging face

→ 1 Million  
 Paid → Context  
 → understanding  
 Go Ahead  
 → free  
 → works

Vector DBs

→ Pine Cone VB

Cloud Based Seamlessly

→ Scales →

Query → Text Query

→ leverage policy

Semantic

2021, 2025, 2024, 2023  
 → leverage policy

BMS

Best Match 26

→ keyword Based

Hybrid RAG

Query

Retrieval 1

Retrieval 2

Semantic Retrieval  
 (VBs - cosine)

Key Word Based  
 (BMS)

Combine → LLM

VBs

D1 - ✓  
 D3 - ✓  
 D4 - ✓  
 D6

D3 - ✓  
 D2 - ✓  
 D1 - ✓  
 D6

# Combining Those Ranks

→ RRF → Reciprocal Rank fusion

